

CSAE 104 — Web Programming

Full-Stack Development Guide

Phase 2: Node.js + Express.js + MongoDB

Complete Installation & Setup Guide for Students with No Backend Experience

Don Mariano Marcos Memorial State University

South La Union Campus · College of Computer Science

From Zero to Full-Stack

VS Code + Node.js +
MongoDB

7 Steps + All Code

Role-Based Access

What This Guide Covers

This guide continues from Phase 1 where your blog site was migrated to React using Create React App (CRA). In Phase 2, you will build a real backend server from scratch and connect it to your React frontend, turning your static blog into a complete working web application.

This guide assumes you have never set up a backend before. Every installation step is explained, and every command is shown exactly as you should type it.

Role	What They Can Do
Guest	Browse and read all published posts. Register for a member account.
Member	Login/logout. Write, edit, delete own posts. Reply to posts. Update profile name, bio, and profile picture. Change password.
Admin	All member abilities. Upload images to posts. Manage members (activate/deactivate). Remove any inappropriate post.

Read This Before You Begin

You need TWO terminal windows open at the same time during development.

Terminal 1 runs your backend server (Express, port 5000).

Terminal 2 runs your React frontend (CRA, port 3000).

Do NOT close either terminal while working. Both must be running together.

Before You Start — Software to Install

All of the software below is free. You only need to install each one once. If you already have it installed from Phase 1, skip to the version check.

Software	What It Does	Where to Download
Visual Studio Code	Your code editor — everything is written here	https://code.visualstudio.com
Node.js (LTS v18+)	Runs JavaScript on your computer (outside the browser)	https://nodejs.org → click LTS
Git	Version control — save and track your code history	https://git-scm.com
MongoDB Community	Your local database server	https://www.mongodb.com/try/download/community
MongoDB Compass	Visual tool to see your database data	Comes bundled with MongoDB Community
Thunder Client	VS Code extension to test your API (like Postman)	Install inside VS Code Extensions panel

Step A — Install Visual Studio Code

1. Go to <https://code.visualstudio.com>
2. Click the big blue Download button. Choose your OS (Windows / macOS).
3. Run the installer. Accept all defaults. Check 'Add to PATH' if the option appears.
4. Open VS Code. You should see the Welcome tab.

Step B — Install Node.js

Node.js lets you run JavaScript on your computer like a server. npm (Node Package Manager) comes automatically with Node.js and lets you install packages like Express.

1. Go to <https://nodejs.org>
2. Click the LTS button (Long Term Support — more stable). Download and run the installer.
3. Accept all defaults during installation. Do NOT uncheck any options.
4. After installing, open a new terminal in VS Code: menu → Terminal → New Terminal
5. Type these commands to verify the installation:

```
node --version
# Expected output: v18.x.x (or higher)

npm --version
# Expected output: 9.x.x (or higher)
```

If 'node' is not recognized

Close VS Code completely and re-open it. The PATH variable updates only after restart.
 If it still fails, uninstall Node.js and reinstall it. Check 'Add to PATH' during setup.
 On Windows: search 'Environment Variables' → Path → confirm C:\Program Files\nodejs\ is listed.

Step C — Install MongoDB Community Server

MongoDB is your database. Unlike MySQL, MongoDB stores data as JSON-like documents (called collections) instead of tables. You will install it on your local machine so you can work without internet.

1. Go to <https://www.mongodb.com/try/download/community>
2. Select: Version = Latest, Platform = Windows (or macOS), Package = MSI
3. Click Download and run the installer.
4. Choose 'Complete' installation type.
5. On the 'Service Configuration' screen — leave all defaults. MongoDB will run as a background service automatically.
6. On the 'Install MongoDB Compass' screen — keep the checkbox checked. Compass is the visual tool for your database.
7. Finish the installation.

✓ Verify MongoDB is Running

Open a new terminal in VS Code and type: `mongosh`
 You should see: 'Connecting to: mongodb://127.0.0.1:27017' and a prompt like: `test>`
 Type: `exit` to quit the shell.
 If `mongosh` is not found, MongoDB may not have been added to PATH — restart your computer first.

Step D — Install VS Code Extensions

Extensions add extra features to VS Code. Install all four below:

8. In VS Code, click the Extensions icon in the left sidebar (looks like four squares), OR press `Ctrl+Shift+X`.
9. Search for each extension by name and click Install:

Extension Name	What It Helps With
Thunder Client	Test your API routes directly inside VS Code (no need for Postman)
ES7+ React/Redux/React-Native Snippets	Shortcut commands for writing React code faster
Prettier - Code Formatter	Auto-formats your code when you save
ESLint	Highlights code errors and bad practices in real time

Set Prettier as Default Formatter

Press `Ctrl+Shift+P` → type 'Format Document With' → select Prettier.

Also enable: File → Preferences → Settings → search 'format on save' → check the box.
Now your code auto-formats every time you press Ctrl+S.

Step E — Verify Everything Before Continuing

Open a terminal (Ctrl+`) in VS Code and run each command. All must work before you continue:

```
node --version      # Should show v18.x.x or higher
npm --version       # Should show 9.x.x or higher
mongosh --version   # Should show 2.x.x or higher
git --version        # Should show git version 2.x.x
```

TASK 0: Pre-Check Screenshot

Take a screenshot of your terminal showing all four commands and their outputs.
All four must show a version number — no errors allowed.
Submit this screenshot before proceeding to Step 1.

How the Full-Stack App Works

Before writing code, understand how the pieces connect. Study this diagram carefully — refer back to it whenever you get confused about where something belongs.



INFO: Why two servers?

React (port 3000) serves your UI — the HTML, CSS, and JavaScript your users see.
 Express (port 5000) is your API server — it handles data, authentication, and file uploads.
 React sends requests to Express using Axios. Express reads/writes to MongoDB.
 In production you would combine them, but during development they run separately.

Project Folder Structure

You will have two main folders. Your Phase 1 React project becomes the 'frontend' folder. You will create the 'backend' folder fresh in this guide.

```

thefolio-project/
├── backend/                                     ← Create this fresh (Phase 2)
│   ├── config/
│   │   └── db.js                               ← MongoDB connection
│   ├── middleware/
│   │   ├── auth.middleware.js                 ← Checks JWT token on requests
│   │   ├── role.middleware.js                 ← Checks admin vs member role
│   │   └── upload.js                           ← Multer image upload config
│   ├── models/
│   │   ├── User.js                           ← User schema
│   │   ├── Post.js                           ← Post schema
│   │   └── Comment.js                         ← Comment schema
│   ├── routes/
│   │   ├── auth.routes.js                     ← /api/auth/*
│   │   ├── post.routes.js                     ← /api/posts/*
│   │   ├── comment.routes.js                 ← /api/comments/*
│   │   └── admin.routes.js                   ← /api/admin/*
│   ├── uploads/                               ← Image files saved here
│   ├── seedAdmin.js                           ← One-time script to create admin
│   ├── .env                                   ← Secret config (NEVER commit this!)
│   ├── .gitignore
│   └── server.js                             ← Main entry point
├── frontend/                                  ← Your Phase 1 React project
│   ├── public/
│   └── src/
│       ├── api/
│       │   └── axios.js                       ← Axios configured for backend
│       ├── context/
│       │   └── AuthContext.js                 ← Global login state
│       ├── components/
│       │   ├── Navbar.js
│       │   ├── PostCard.js
│       │   └── ProtectedRoute.js             ← Blocks non-logged-in users
│       ├── pages/
│       │   ├── HomePage.js                   ← All published posts
│       │   ├── PostPage.js                   ← Single post + comments
│       │   ├── LoginPage.js
│       │   ├── RegisterPage.js
│       │   ├── ProfilePage.js                ← Edit profile, change password
│       │   ├── CreatePostPage.js
│       │   ├── EditPostPage.js
│       │   └── AdminPage.js                  ← Admin dashboard

```

STEP
1

Create the Backend — Folder, npm & Packages

Concept: `npm init`, `Express`, `Mongoose`, `environment setup`

Estimated Time: 30–45 minutes

1.1 — Open VS Code in Your Project Folder

1. Open VS Code.
2. Go to File → Open Folder. Navigate to where you want your project to live (e.g., Desktop or Documents).
3. Create a new folder called thefolio-project. Open it.
4. Open the integrated terminal: menu → Terminal → New Terminal (or press `Ctrl+``).

1.2 — Create and Initialize the Backend

In the VS Code terminal, type these commands one by one. Press Enter after each line:

```
# Create the backend folder
mkdir backend

# Go inside it
cd backend

# Initialize a new Node.js project (creates package.json)
npm init -y
```

INFO: What does 'npm init -y' do?

It creates a `package.json` file — the ID card of your Node.js project.
 The `-y` flag answers 'yes' to all questions so it runs without prompts.
`package.json` records your project name, version, and list of installed packages.

1.3 — Install All Required npm Packages

This is the most important install command. Copy it exactly as shown:

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cors multer

npm install --save-dev nodemon
```

Wait for installation to finish. You will see a 'added N packages' message when done.

Package	What It Does in This Project
express	Creates your web server and defines API routes
mongoose	Connects to MongoDB and lets you define schemas (data shapes)
dotenv	Reads your <code>.env</code> file so secrets stay out of your

	code
bcryptjs	Hashes (encrypts) passwords before saving them to the database
jsonwebtoken	Creates JWT tokens so users stay logged in across page refreshes
cors	Allows React (port 3000) to call Express (port 5000) — without this, the browser blocks the request
multer	Handles image file uploads from forms (profile pictures, post images)
nodemon	Dev tool: watches your files and restarts the server automatically when you save

1.4 — Create the .env File

The .env file holds your secret configuration. Create it in the backend/ folder:

1. In VS Code's Explorer panel (left sidebar), right-click the backend/ folder → New File.
2. Name it exactly: .env (with the dot, no extension after)
3. Paste this content:

```
# backend/.env

MONGO_URI=mongodb://127.0.0.1:27017/thefolio
JWT_SECRET=thefolio_super_secret_key_change_this
PORT=5000
```

INFO: MONGO_URI explained

mongodb://127.0.0.1:27017 → connects to MongoDB on your own computer (localhost)
 /thefolio → the name of your database (MongoDB creates it automatically on first use)
 You do NOT need to create the database manually — Mongoose does it for you.

IMPORTANT: Add .env to .gitignore

Create a file named .gitignore in the backend/ folder.
 Add this line inside it: node_modules/
 Then add a second line: .env
 This prevents your secret keys and packages from being uploaded to GitHub.

1.5 — Create config/db.js

First, create the config/ folder inside backend/. Then create db.js inside it:

```
// backend/config/db.js
const mongoose = require('mongoose');
```

```
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error('Database connection error:', error.message);
    process.exit(1); // Stop the server if DB fails
  }
};

module.exports = connectDB;
```

1.6 — Create server.js

Create server.js in the root of your backend/ folder (not inside any subfolder):

```
// backend/server.js
require('dotenv').config(); // Load .env variables FIRST
const express = require('express');
const cors = require('cors');
const path = require('path');
const connectDB = require('./config/db');

// Import routes (you will create these files in the next steps)
const authRoutes = require('./routes/auth.routes');
const postRoutes = require('./routes/post.routes');
const commentRoutes = require('./routes/comment.routes');
const adminRoutes = require('./routes/admin.routes');

const app = express();
connectDB(); // Connect to MongoDB

// — Middleware —
// Allow React (port 3000) to call this server
app.use(cors({ origin: 'http://localhost:3000', credentials: true }));

// Parse incoming JSON request bodies
app.use(express.json());

// Serve uploaded image files as public URLs
// e.g. http://localhost:5000/uploads/my-image.jpg
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// — Routes —
app.use('/api/auth', authRoutes);
app.use('/api/posts', postRoutes);
app.use('/api/comments', commentRoutes);
app.use('/api/admin', adminRoutes);

// — Start Server —
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

1.7 — Add npm Scripts to package.json

Open backend/package.json. Find the 'scripts' section and replace it with:

```
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

You cannot start the server yet

server.js imports the route files (auth.routes.js, etc.) that you have not created yet. Node.js will throw an error if those files are missing. Complete Steps 2, 3, and 4 first — then start the server.

TASK 1: Folder Check

Your backend/ folder should now contain: server.js, .env, .gitignore, config/db.js, package.json. Take a screenshot of your VS Code Explorer panel showing the backend/ file tree. Submit before moving to Step 2.

STEP
2**Database Models — Schemas with Mongoose**

Concept: Mongoose Schema, pre-save hooks, model methods

Estimated Time: 30–40 minutes

A Mongoose schema defines the shape of your data — what fields each document has, their data types, and rules. Each schema becomes a MongoDB collection.

2.1 — Create the models/ Folder

In VS Code Explorer, right-click backend/ → New Folder → name it: models

2.2 — User Model (models/User.js)

```
// backend/models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6 },
  role: { type: String, enum: ['member', 'admin'], default: 'member' },
  status: { type: String, enum: ['active', 'inactive'], default:
'active' },
  bio: { type: String, default: '' },
  profilePic: { type: String, default: '' }, // stores filename e.g.
'abc123.jpg'
}, { timestamps: true }); // adds createdAt and updatedAt automatically

// — Pre-save hook: hash password before storing —————
// This runs automatically every time you call user.save()
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next(); // skip if password
  unchanged
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// — Instance method: compare entered password with stored hash —
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

2.3 — Post Model (models/Post.js)

```
// backend/models/Post.js
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  body: { type: String, required: true },
  image: { type: String, default: '' }, // filename stored in uploads/
```

```
    author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
    status: { type: String, enum: ['published', 'removed'], default:
'published' },
  }, { timestamps: true });

module.exports = mongoose.model('Post', postSchema);
```

2.4 — Comment Model (models/Comment.js)

```
// backend/models/Comment.js
const mongoose = require('mongoose');

const commentSchema = new mongoose.Schema({
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post', required:
true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  body: { type: String, required: true, trim: true },
}, { timestamps: true });

module.exports = mongoose.model('Comment', commentSchema);
```

INFO: What is 'ref' and why use it?

{ ref: 'User' } creates a relationship between collections — like a foreign key in MySQL.
author: { ref: 'User' } means: store the User's `_id` in this field.
Later you can call `.populate('author')` to replace that `_id` with the full user object.
Example: instead of showing `authorId: '64abc...'`, you get `author: { name: 'Juan', email: '...' }`

STEP
3

Middleware — JWT Auth & Role Checking

Concept: JWT verification, role-based access control, Multer upload

Estimated Time: 30–40 minutes

Middleware functions run between the request arriving and your route handler executing. Think of them as guards at the door — they check something and either let the request through or send back an error.

3.1 — Create the middleware/ Folder

Right-click backend/ → New Folder → name it: middleware

3.2 — Auth Middleware (middleware/auth.middleware.js)

This checks that a valid JWT token is present in the request header. Add this to any route that requires login.

```
// backend/middleware/auth.middleware.js
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;

  // Look for 'Authorization: Bearer <token>' in request headers
  if (req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer ')) {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return res.status(401).json({ message: 'Not authorized - please log in first' });
  }

  try {
    // Verify the token using your JWT_SECRET
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Attach the full user object to req.user (minus the password)
    req.user = await User.findById(decoded.id).select('-password');

    if (!req.user || req.user.status === 'inactive') {
      return res.status(401).json({ message: 'Account not found or deactivated' });
    }

    next(); // Pass to the next handler
  } catch (err) {
    return res.status(401).json({ message: 'Token is invalid or has expired' });
  }
};

module.exports = { protect };
```

3.3 — Role Middleware (middleware/role.middleware.js)

```
// backend/middleware/role.middleware.js

// Only admins can pass
const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'admin') return next();
  return res.status(403).json({ message: 'Access denied - Admins only' });
};

// Members or Admins can pass (but not guests/unauthenticated users)
const memberOrAdmin = (req, res, next) => {
  if (req.user && (req.user.role === 'member' || req.user.role === 'admin'))
    return next();
  return res.status(403).json({ message: 'Access denied - Members only' });
};

module.exports = { adminOnly, memberOrAdmin };
```

3.4 — Upload Middleware (middleware/upload.js)

Multer handles file uploads. This configuration saves images to your uploads/ folder with unique filenames.

```
// backend/middleware/upload.js
const multer = require('multer');
const path = require('path');
const fs = require('fs');

// Create uploads/ folder if it does not exist yet
if (!fs.existsSync('uploads')) {
  fs.mkdirSync('uploads');
}

// Where and how to save uploaded files
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => {
    // Create a unique filename: timestamp + random number + original extension
    const unique = Date.now() + '-' + Math.round(Math.random() * 1e9);
    cb(null, unique + path.extname(file.originalname));
    // Example result: 1719123456789-342156789.jpg
  },
});

// Only allow image file types
const fileFilter = (req, file, cb) => {
  const allowedTypes = /jpeg|jpg|png|gif|webp/;
  const ext =
    allowedTypes.test(path.extname(file.originalname).toLowerCase());
  const mime = allowedTypes.test(file.mimetype);
  if (ext && mime) return cb(null, true);
  cb(new Error('Only image files are allowed (jpg, png, gif, webp)'));
};

const upload = multer({
  storage,
  fileFilter,
  limits: { fileSize: 5 * 1024 * 1024 }, // max 5 MB per file
});
```

```
module.exports = upload;
```


STEP
4

API Routes — Auth, Posts, Comments & Admin

Concept: REST API, CRUD operations, populate(), role protection

Estimated Time: 60–90 minutes

Create a routes/ folder inside backend/. You will create four files inside it.

4.1 — Auth Routes (routes/auth.routes.js)

```
// backend/routes/auth.routes.js
const express = require('express');
const jwt      = require('jsonwebtoken');
const User     = require('../models/User');
const { protect } = require('../middleware/auth.middleware');
const upload    = require('../middleware/upload');
const router    = express.Router();

// Helper function — generates a JWT token that expires in 7 days
const generateToken = (id) => jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: '7d' });

// — POST /api/auth/register —————
router.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  try {
    const exists = await User.findOne({ email });
    if (exists) return res.status(400).json({ message: 'Email is already registered' });
    const user = await User.create({ name, email, password });
    res.status(201).json({
      token: generateToken(user._id),
      user: { _id: user._id, name: user.name, email: user.email, role: user.role }
    });
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// — POST /api/auth/login —————
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ email });
    if (!user) return res.status(400).json({ message: 'Invalid email or password' });
    if (user.status === 'inactive')
      return res.status(403).json({ message: 'Your account is deactivated. Please contact the admin.' });
    const match = await user.matchPassword(password);
    if (!match) return res.status(400).json({ message: 'Invalid email or password' });
    res.json({
      token: generateToken(user._id),
      user: { _id: user._id, name: user.name, email: user.email, role: user.role, profilePic: user.profilePic }
    });
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// — GET /api/auth/me —————
// Returns the currently logged-in user's data (requires token)
router.get('/me', protect, async (req, res) => {
  const user = await User.findById(req.user._id).select('-password');
```

```

    res.json(user);
  });

// — PUT /api/auth/profile —————
// Update name, bio, or upload a new profile picture
router.put('/profile', protect, upload.single('profilePic'), async (req, res)
=> {
  try {
    const user = await User.findById(req.user._id);
    if (req.body.name) user.name = req.body.name;
    if (req.body.bio) user.bio = req.body.bio;
    if (req.file) user.profilePic = req.file.filename;
    await user.save();
    const updated = await User.findById(user._id).select('-password');
    res.json(updated);
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// — PUT /api/auth/change-password —————
router.put('/change-password', protect, async (req, res) => {
  const { currentPassword, newPassword } = req.body;
  try {
    const user = await User.findById(req.user._id);
    const match = await user.matchPassword(currentPassword);
    if (!match) return res.status(400).json({ message: 'Current password is
incorrect' });
    user.password = newPassword; // pre-save hook will hash this
    await user.save();
    res.json({ message: 'Password updated successfully' });
  } catch (err) { res.status(500).json({ message: err.message }); }
});

module.exports = router;

```

4.2 — Post Routes (routes/post.routes.js)

```

// backend/routes/post.routes.js
const express = require('express');
const Post = require('../models/Post');
const { protect } = require('../middleware/auth.middleware');
const { memberOrAdmin } = require('../middleware/role.middleware');
const upload = require('../middleware/upload');
const router = express.Router();

// GET /api/posts — Public: all published posts (newest first)
router.get('/', async (req, res) => {
  try {
    const posts = await Post.find({ status: 'published' })
      .populate('author', 'name profilePic')
      .sort({ createdAt: -1 });
    res.json(posts);
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// GET /api/posts/:id — Public: single post by ID
router.get('/:id', async (req, res) => {
  try {
    const post = await Post.findById(req.params.id).populate('author', 'name
profilePic');
    if (!post || post.status === 'removed')
      return res.status(404).json({ message: 'Post not found' });
  }

```

```

        res.json(post);
    } catch (err) { res.status(500).json({ message: err.message }); }
});

// POST /api/posts - Member or Admin: create new post
// upload.single('image') handles optional image file
router.post('/', protect, memberOrAdmin, upload.single('image'), async (req,
res) => {
    try {
        const { title, body } = req.body;
        const image = req.file ? req.file.filename : '';
        const post = await Post.create({ title, body, image, author:
req.user._id });
        await post.populate('author', 'name profilePic');
        res.status(201).json(post);
    } catch (err) { res.status(500).json({ message: err.message }); }
});

// PUT /api/posts/:id - Edit: only post owner OR admin
router.put('/:id', protect, memberOrAdmin, upload.single('image'), async (req,
res) => {
    try {
        const post = await Post.findById(req.params.id);
        if (!post) return res.status(404).json({ message: 'Post not found' });

        const isOwner = post.author.toString() === req.user._id.toString();
        const isAdmin = req.user.role === 'admin';
        if (!isOwner && !isAdmin) return res.status(403).json({ message: 'Not
authorized' });

        if (req.body.title) post.title = req.body.title;
        if (req.body.body) post.body = req.body.body;
        if (req.file) post.image = req.file.filename;
        await post.save();
        res.json(post);
    } catch (err) { res.status(500).json({ message: err.message }); }
});

// DELETE /api/posts/:id - Delete: only post owner OR admin
router.delete('/:id', protect, memberOrAdmin, async (req, res) => {
    try {
        const post = await Post.findById(req.params.id);
        if (!post) return res.status(404).json({ message: 'Post not found' });

        const isOwner = post.author.toString() === req.user._id.toString();
        const isAdmin = req.user.role === 'admin';
        if (!isOwner && !isAdmin) return res.status(403).json({ message: 'Not
authorized' });

        await post.deleteOne();
        res.json({ message: 'Post deleted successfully' });
    } catch (err) { res.status(500).json({ message: err.message }); }
});

module.exports = router;

```

4.3 — Comment Routes (routes/comment.routes.js)

```

// backend/routes/comment.routes.js
const express = require('express');

```

```

const Comment = require('../models/Comment');
const { protect } = require('../middleware/auth.middleware');
const { memberOrAdmin } = require('../middleware/role.middleware');
const router = express.Router();

// GET /api/comments/:postId - Public: all comments for a post
router.get('/:postId', async (req, res) => {
  try {
    const comments = await Comment.find({ post: req.params.postId })
      .populate('author', 'name profilePic')
      .sort({ createdAt: 1 }); // oldest first
    res.json(comments);
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// POST /api/comments/:postId - Member/Admin: add a comment
router.post('/:postId', protect, memberOrAdmin, async (req, res) => {
  try {
    const comment = await Comment.create({
      post: req.params.postId,
      author: req.user._id,
      body: req.body.body,
    });
    await comment.populate('author', 'name profilePic');
    res.status(201).json(comment);
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// DELETE /api/comments/:id - Own comment OR admin
router.delete('/:id', protect, memberOrAdmin, async (req, res) => {
  try {
    const comment = await Comment.findById(req.params.id);
    if (!comment) return res.status(404).json({ message: 'Comment not found' });

    const isOwner = comment.author.toString() === req.user._id.toString();
    const isAdmin = req.user.role === 'admin';
    if (!isOwner && !isAdmin) return res.status(403).json({ message: 'Not authorized' });

    await comment.deleteOne();
    res.json({ message: 'Comment deleted' });
  } catch (err) { res.status(500).json({ message: err.message }); }
});

module.exports = router;

```

4.4 — Admin Routes (routes/admin.routes.js)

```

// backend/routes/admin.routes.js
const express = require('express');
const User = require('../models/User');
const Post = require('../models/Post');
const { protect } = require('../middleware/auth.middleware');
const { adminOnly } = require('../middleware/role.middleware');
const router = express.Router();

// All routes below require: (1) valid token AND (2) admin role
router.use(protect, adminOnly);

```

```
// GET /api/admin/users - List all non-admin members
router.get('/users', async (req, res) => {
  try {
    const users = await User.find({ role: { $ne: 'admin' } })
      .select('-password')
      .sort({ createdAt: -1 });
    res.json(users);
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// PUT /api/admin/users/:id/status - Toggle member active/inactive
router.put('/users/:id/status', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user || user.role === 'admin')
      return res.status(404).json({ message: 'User not found' });
    user.status = user.status === 'active' ? 'inactive' : 'active';
    await user.save();
    res.json({ message: `User is now ${user.status}`, user });
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// GET /api/admin/posts - List ALL posts including removed ones
router.get('/posts', async (req, res) => {
  try {
    const posts = await Post.find()
      .populate('author', 'name email')
      .sort({ createdAt: -1 });
    res.json(posts);
  } catch (err) { res.status(500).json({ message: err.message }); }
});

// PUT /api/admin/posts/:id/remove - Mark post as removed (inappropriate)
router.put('/posts/:id/remove', async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (!post) return res.status(404).json({ message: 'Post not found' });
    post.status = 'removed';
    await post.save();
    res.json({ message: 'Post has been removed', post });
  } catch (err) { res.status(500).json({ message: err.message }); }
});

module.exports = router;
```

STEP
5**Create Admin Account & Start the Server****Concept:** Seeding database, running nodemon, testing with Thunder Client
Estimated Time: 20–30 minutes

5.1 — Create the Admin Seed Script

The admin account is never created through the registration page. Create this one-time script in the backend/ root folder:

```
// backend/seedAdmin.js
require('dotenv').config();
const connectDB = require('./config/db');
const User = require('./models/User');

connectDB().then(async () => {
  const exists = await User.findOne({ email: 'admin@thefolio.com' });
  if (exists) {
    console.log('Admin account already exists.');
    process.exit();
  }
  await User.create({
    name: 'TheFolio Admin',
    email: 'admin@thefolio.com',
    password: 'Admin@1234',
    role: 'admin',
  });
  console.log('Admin account created successfully!');
  console.log('Email: admin@thefolio.com');
  console.log('Password: Admin@1234');
  process.exit();
});
```

5.2 — Run the Seed Script

In your VS Code terminal (make sure you are in the backend/ folder):

```
node seedAdmin.js

# Expected output:
# MongoDB Connected: 127.0.0.1
# Admin account created successfully!
# Email: admin@thefolio.com
# Password: Admin@1234
```

Delete seedAdmin.js after running it

Once the admin is created, delete seedAdmin.js — you only need it once. Leaving it in your project is not dangerous locally, but it's good practice to remove it.

5.3 — Start Your Backend Server

```
# Make sure you are in the backend/ folder
```

```
npm run dev

# Expected output (in green):
# [nodemon] starting 'node server.js'
# Server is running on http://localhost:5000
# MongoDB Connected: 127.0.0.1
```

✔ Your backend is now running!

Leave this terminal open. Do not close it.
 Every time you save a file, nodemon automatically restarts the server.
 If you see any red errors, check the error message and compare your code carefully.

5.4 — Test Your API with Thunder Client

Thunder Client is the VS Code extension you installed earlier. Click its icon in the left sidebar (looks like a lightning bolt).

Test #	Method	URL	Body / Header	Expected Result
1	POST	http://localhost:5000/api/auth/register	JSON: { name, email, password }	201 — returns token + user object
2	POST	http://localhost:5000/api/auth/login	JSON: { email, password }	200 — returns token + user object
3	GET	http://localhost:5000/api/auth/me	Header: Authorization: Bearer <your_token>	200 — returns your user profile
4	POST	http://localhost:5000/api/auth/login	Admin: admin@thefolio.com / Admin@1234	200 — role should be 'admin'

How to add Authorization header in Thunder Client

After logging in, copy the token value from the response.
 In your next request, click the 'Auth' tab → select 'Bearer Token'.
 Paste your token there. Thunder Client adds the header automatically.

TASK 5: API Test Screenshots

Run all 4 Thunder Client tests listed above.
 Screenshot each response — show the status code (200/201) and the JSON body.
 Also open MongoDB Compass → connect to mongodb://localhost:27017 → open thefolio database.
 Screenshot the 'users' collection — you should see your test user and the admin account.

STEP
6

Connect the React Frontend

Concept: Axios, AuthContext, ProtectedRoute, App.js routing

Estimated Time: 45–60 minutes

Now you connect your Phase 1 React project to the backend. Open a **SECOND** terminal in VS Code (click the + icon in the terminal panel) and navigate to your frontend folder:

```
# In the SECOND terminal
cd frontend
npm install axios
```

6.1 — Create src/api/axios.js

Create a new folder inside src/ called api/. Then create axios.js inside it:

```
// frontend/src/api/axios.js
import axios from 'axios';

const instance = axios.create({
  baseURL: 'http://localhost:5000/api',
});

// This interceptor runs before EVERY request.
// It reads the token from localStorage and adds it to the Authorization
// header.
instance.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default instance;
```

6.2 — Create src/context/AuthContext.js

Create a context/ folder inside src/. Then create AuthContext.js. This stores the logged-in user globally so any page or component can access it.

```
// frontend/src/context/AuthContext.js
import { createContext, useContext, useState, useEffect } from 'react';
import API from '../api/axios';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // On app load: if a token exists in localStorage, fetch the user's data
  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
      API.get('/auth/me')
    }
  }, []);

  return (
    <AuthContext.Provider value={{ user, setUser, loading, setLoading }}>
      {children}
    </AuthContext.Provider>
  );
};
```



```

        .then(res => setUser(res.data))
        .catch(() => localStorage.removeItem('token')) // remove bad token
        .finally(() => setLoading(false));
    } else {
        setLoading(false);
    }
  }, []);

// login(): call the backend, save token, store user in state
const login = async (email, password) => {
  const { data } = await API.post('/auth/login', { email, password });
  localStorage.setItem('token', data.token);
  setUser(data.user);
  return data.user; // return user so caller can check role
};

// logout(): clear token and user from memory
const logout = () => {
  localStorage.removeItem('token');
  setUser(null);
};

return (
  <AuthContext.Provider value={{ user, setUser, login, logout, loading }}>
    {!loading && children}
  </AuthContext.Provider>
);
};

// Custom hook — use this instead of useContext(AuthContext) everywhere
export const useAuth = () => useContext(AuthContext);

```

6.3 — Update src/index.js

Wrap the app with AuthProvider and BrowserRouter so both are available everywhere:

```

// frontend/src/index.js
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import App from './App';
import './index.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <BrowserRouter>
    <AuthProvider>
      <App />
    </AuthProvider>
  </BrowserRouter>
);

```

6.4 — Create ProtectedRoute Component

Create src/components/ProtectedRoute.js — this blocks pages from unauthenticated or wrong-role users:

```
// frontend/src/components/ProtectedRoute.js
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

// Usage:
// <ProtectedRoute>                                → requires any logged-in user
// <ProtectedRoute role='admin'> → requires admin role
const ProtectedRoute = ({ children, role }) => {
  const { user } = useAuth();
  if (!user) return <Navigate to='/login' replace />;
  if (role && user.role !== role) return <Navigate to='/' replace />;
  return children;
};

export default ProtectedRoute;
```

6.5 — Update App.js with All Routes

```
// frontend/src/App.js
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import ProtectedRoute from './components/ProtectedRoute';
import SplashPage from './pages/SplashPage';
import HomePage from './pages/HomePage';
import PostPage from './pages/PostPage';
import LoginPage from './pages/LoginPage';
import RegisterPage from './pages/RegisterPage';
import ProfilePage from './pages/ProfilePage';
import CreatePostPage from './pages/CreatePostPage';
import EditPostPage from './pages/EditPostPage';
import AdminPage from './pages/AdminPage';

function App() {
  return (
    <>
      <Navbar />
      <Routes>
        {/* Public routes - anyone can visit */}
        <Route path='/' element={<SplashPage />} />
        <Route path='/home' element={<HomePage />} />
        <Route path='/posts/:id' element={<PostPage />} />
        <Route path='/login' element={<LoginPage />} />
        <Route path='/register' element={<RegisterPage />} />

        {/* Protected routes - must be logged in */}
        <Route path='/profile'
          element={<ProtectedRoute><ProfilePage /></ProtectedRoute>} />
        <Route path='/create-post'
          element={<ProtectedRoute><CreatePostPage /></ProtectedRoute>} />
        <Route path='/edit-post/:id'
          element={<ProtectedRoute><EditPostPage /></ProtectedRoute>} />

        {/* Admin only - redirects members/guests to home */}
        <Route path='/admin'
          element={<ProtectedRoute role='admin'><AdminPage /></ProtectedRoute>}
      />
    </Routes>
  </>
);
}
```

```
export default App;
```

STEP
7

Build the React Pages

Concept: `useState`, `useEffect`, `Axios` API calls, `FormData`, conditional rendering
Estimated Time: 90–120 minutes

Create all new pages in `src/pages/`. Each page communicates with the backend through the configured `Axios` instance.

7.1 — LoginPage.js

```
// frontend/src/pages/LoginPage.js
import { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const LoginPage = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    try {
      const user = await login(email, password);
      // Redirect based on role
      navigate(user.role === 'admin' ? '/admin' : '/home');
    } catch (err) {
      setError(err.response?.data?.message || 'Login failed. Please try again.');
```

7.2 — RegisterPage.js

```
// frontend/src/pages/RegisterPage.js
import { useState } from 'react';
```

```

import { useNavigate, Link } from 'react-router-dom';
import API from '../api/axios';

const RegisterPage = () => {
  const [form, setForm] = useState({ name: '', email: '', password: '' });
  const [error, setError] = useState('');
  const navigate = useNavigate();

  const handleChange = (e) => setForm({ ...form, [e.target.name]:
e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault(); setError('');
    try {
      const { data } = await API.post('/auth/register', form);
      localStorage.setItem('token', data.token);
      navigate('/home');
    } catch (err) {
      setError(err.response?.data?.message || 'Registration failed.');

```

7.3 — HomePage.js

```

// frontend/src/pages/HomePage.js
import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import API from '../api/axios';

const HomePage = () => {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    API.get('/posts')
      .then(res => setPosts(res.data))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading posts...</p>;

```

```

    return (
      <div className='home-page'>
        <h2>Latest Posts</h2>
        {posts.length === 0 && <p>No posts yet. Be the first to write one!</p>}
        <div className='posts-grid'>
          {posts.map(post => (
            <div key={post._id} className='post-card'>
              {post.image && (
                <img src={`http://localhost:5000/uploads/${post.image}`}
alt={post.title} />
              )}
              <h3><Link to={`/${post._id}`}>{post.title}</Link></h3>
              <p>{post.body.substring(0, 120)}...</p>
              <small>By {post.author?.name} · {new
Date(post.createdAt).toLocaleDateString()}</small>
            </div>
          ))}
        </div>
      </div>
    );
  };
};

export default HomePage;

```

7.4 — ProfilePage.js

```

// frontend/src/pages/ProfilePage.js
import { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import API from '../api/axios';

const ProfilePage = () => {
  const { user, setUser } = useAuth();
  const [name, setName] = useState(user?.name || '');
  const [bio, setBio] = useState(user?.bio || '');
  const [pic, setPic] = useState(null);
  const [curPw, setCurPw] = useState('');
  const [newPw, setNewPw] = useState('');
  const [msg, setMsg] = useState('');

  const handleProfile = async (e) => {
    e.preventDefault(); setMsg('');
    const fd = new FormData();
    fd.append('name', name);
    fd.append('bio', bio);
    if (pic) fd.append('profilePic', pic);
    try {
      // Do NOT manually set Content-Type — Axios sets multipart automatically
      const { data } = await API.put('/auth/profile', fd);
      setUser(data);
      setMsg('Profile updated successfully!');
    } catch (err) { setMsg(err.response?.data?.message || 'Error'); }
  };

  const handlePassword = async (e) => {
    e.preventDefault(); setMsg('');
    try {
      await API.put('/auth/change-password', { currentPassword: curPw,
newPassword: newPw });
      setMsg('Password changed successfully!');
    }
  };

```

```

        setCurPw(''); setNewPw('');
    } catch (err) { setMsg(err.response?.data?.message || 'Error'); }
};

const picSrc = user?.profilePic
  ? `http://localhost:5000/uploads/${user.profilePic}`
  : '/default-avatar.png';

return (
  <div className='profile-page'>
    <h2>My Profile</h2>
    <img src={picSrc} alt='Profile' className='profile-pic-preview' />
    {msg && <p className='success-msg'>{msg}</p>}

    <form onSubmit={handleProfile}>
      <h3>Edit Profile</h3>
      <input value={name} onChange={e => setName(e.target.value)}
placeholder='Display name' />
      <textarea value={bio} onChange={e => setBio(e.target.value)}
placeholder='Short bio...' rows={3} />
      <label>Change Profile Picture:</label>
      <input type='file' accept='image/*' onChange={e =>
setPic(e.target.files[0])} />
      <button type='submit'>Save Profile</button>
    </form>

    <form onSubmit={handlePassword}>
      <h3>Change Password</h3>
      <input type='password' placeholder='Current password'
value={curPw} onChange={e => setCurPw(e.target.value)} required />
      <input type='password' placeholder='New password (min 6 chars)'
value={newPw} onChange={e => setNewPw(e.target.value)} required
minLength={6} />
      <button type='submit'>Change Password</button>
    </form>
  </div>
);
};

export default ProfilePage;

```

7.5 — CreatePostPage.js

```

// frontend/src/pages/CreatePostPage.js
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import API from '../api/axios';

const CreatePostPage = () => {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
  const [image, setImage] = useState(null);
  const [error, setError] = useState('');
  const { user } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault(); setError('');
    const fd = new FormData();

```

```

    fd.append('title', title);
    fd.append('body', body);
    if (image) fd.append('image', image);
    try {
      const { data } = await API.post('/posts', fd);
      navigate(`/posts/${data._id}`);
    } catch (err) { setError(err.response?.data?.message || 'Failed to publish
post'); }
  };

  return (
    <div className='create-post-page'>
      <h2>Write a New Post</h2>
      {error && <p className='error-msg'>{error}</p>}
      <form onSubmit={handleSubmit}>
        <input value={title} onChange={e => setTitle(e.target.value)}
          placeholder='Post title' required />
        <textarea value={body} onChange={e => setBody(e.target.value)}
          placeholder='Write your post here...' rows={12} required />
        {user?.role === 'admin' && (
          <div>
            <label>Upload Cover Image (Admin only):</label>
            <input type='file' accept='image/*' onChange={e =>
setImage(e.target.files[0])} />
          </div>
        )}
        <button type='submit'>Publish Post</button>
      </form>
    </div>
  );
};

export default CreatePostPage;

```

7.6 — AdminPage.js

```

// frontend/src/pages/AdminPage.js
import { useState, useEffect } from 'react';
import API from '../api/axios';

const AdminPage = () => {
  const [users, setUsers] = useState([]);
  const [posts, setPosts] = useState([]);
  const [tab, setTab] = useState('users');

  useEffect(() => {
    API.get('/admin/users').then(r => setUsers(r.data));
    API.get('/admin/posts').then(r => setPosts(r.data));
  }, []);

  const toggleStatus = async (id) => {
    const { data } = await API.put(`/admin/users/${id}/status`);
    setUsers(users.map(u => u._id === id ? data.user : u));
  };

  const removePost = async (id) => {
    await API.put(`/admin/posts/${id}/remove`);
    setPosts(posts.map(p => p._id === id ? { ...p, status: 'removed' } : p));
  };

```



```

    return (
      <div className='admin-page'>
        <h2>Admin Dashboard</h2>
        <div className='admin-tabs'>
          <button onClick={() => setTab('users')} className={tab === 'users' ?
'active' : ''}>
            Members ({users.length})
          </button>
          <button onClick={() => setTab('posts')} className={tab === 'posts' ?
'active' : ''}>
            All Posts ({posts.length})
          </button>
        </div>

        {tab === 'users' && (
          <table className='admin-table'>

<thead><tr><th>Name</th><th>Email</th><th>Status</th><th>Action</th></tr></thead>
          <tbody>
            {users.map(u => (
              <tr key={u._id}>
                <td>{u.name}</td>
                <td>{u.email}</td>
                <td><span className={`status-badge
${u.status}`}>{u.status}</span></td>
                <td>
                  <button onClick={() => toggleStatus(u._id)}
                    className={u.status === 'active' ? 'btn-danger' : 'btn-
success'}>
                    {u.status === 'active' ? 'Deactivate' : 'Activate'}
                  </button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      )}

      {tab === 'posts' && (
        <table className='admin-table'>

<thead><tr><th>Title</th><th>Author</th><th>Status</th><th>Action</th></tr></thead>
        <tbody>
          {posts.map(p => (
            <tr key={p._id}>
              <td>{p.title}</td>
              <td>{p.author?.name}</td>
              <td><span className={`status-badge
${p.status}`}>{p.status}</span></td>
              <td>
                {p.status === 'published' && (
                  <button className='btn-danger' onClick={() =>
removePost(p._id)}>
                    Remove
                  </button>
                )}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
  </div>
);
};

```

```
export default AdminPage;
```

TASK 7: Full Integration Test

Open Terminal 1 (backend): `npm run dev` — confirm server is running.
Open Terminal 2 (frontend): `npm start` — confirm browser opens at `localhost:3000`.
Register a new member account. Login. Write a post. Leave a comment.
Update your profile — upload a profile picture. Change your password.
Login as admin (`admin@thefolio.com` / `Admin@1234`).
Go to `/admin` → deactivate the test member → try to login as them (should be blocked).
Re-activate the member. Remove one of their posts from the Posts tab.
Confirm the removed post does not appear on the public home page.
Screenshot each step and submit your complete test evidence.

Common Errors & How to Fix Them

Error / Symptom	Likely Cause	Fix
'node' is not recognized	Node.js not added to PATH	Restart VS Code. If still failing, reinstall Node.js — check 'Add to PATH'.
'mongosh' is not recognized	MongoDB not in PATH	Restart your computer. If still failing, add MongoDB bin folder to system PATH manually.
CORS error in browser console	Backend not allowing React's port	Confirm: cors({ origin: 'http://localhost:3000' }) is in server.js
401 Unauthorized	Token missing or expired	Re-login to get a fresh token. Check Axios interceptor in axios.js.
403 Forbidden	Wrong user role	Confirm req.user.role === 'admin'. Check token belongs to admin account.
Cannot read properties of null (reading 'role')	AuthContext not loaded yet	The loading state in AuthContext prevents this. Check index.js wraps App correctly.
Multer: file not saved	Content-Type set manually	Never set Content-Type manually when using FormData. Axios sets multipart/form-data automatically.
Image broken (no image shown)	Wrong static file path	Confirm: app.use('/uploads', express.static(...)) is in server.js. Image URL: http://localhost:5000/uploads/<filename>
MongoServerError: E11000 duplicate key	Email already registered	Show 'Email already in use' message. Check User.findOne({ email }) before creating.
nodemon not found	nodemon not installed as devDependency	Run: npm install --save-dev nodemon inside the backend/ folder.

Daily Quick Start — Both Servers

Every time you sit down to work on this project, open TWO terminals and run these commands:

Terminal 1 — Backend	Terminal 2 — Frontend
<pre>cd thefolio-project/backend npm run dev # Wait for: # Server running on port 5000 # MongoDB Connected: 127.0.0.1</pre>	<pre>cd thefolio-project/frontend npm start # Wait for: # Compiled successfully! # Opened: http://localhost:3000</pre>

If MongoDB is not connecting

MongoDB runs as a background Windows Service — it should start automatically on boot.
If not: press Win key → search 'Services' → find 'MongoDB Server' → right-click → Start.
Or run in a terminal: `net start MongoDB`
On macOS: `brew services start mongodb-community`

API Reference — All Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new member account
POST	/api/auth/login	Public	Login — returns JWT token
GET	/api/auth/me	Protected	Get current user's profile
PUT	/api/auth/profile	Protected	Update name, bio, and/or profile picture
PUT	/api/auth/change-password	Protected	Change own password
GET	/api/posts	Public	Get all published posts
GET	/api/posts/:id	Public	Get one post by its MongoDB ID
POST	/api/posts	Member/Admin	Create new post (with optional image)
PUT	/api/posts/:id	Member/Admin	Edit own post (admin: edit any)
DELETE	/api/posts/:id	Member/Admin	Delete own post (admin: delete any)
GET	/api/comments/:postId	Public	Get all comments for a post
POST	/api/comments/:postId	Member/Admin	Add a reply to a post
DELETE	/api/comments/:id	Member/Admin	Delete own comment (admin: any)
GET	/api/admin/users	Admin only	List all member accounts
PUT	/api/admin/users/:id/status	Admin only	Toggle member active/inactive
GET	/api/admin/posts	Admin only	List all posts including removed
PUT	/api/admin/posts/:id/remove	Admin only	Mark post as removed/inappropriate

Final Project Grading Rubric

Criterion	Points	What Will Be Checked
Installation & Setup	10	All software installed and verified. Backend folder structure correct.
Authentication	20	Register, login, logout functional. JWT verified. Inactive users cannot login.
Post CRUD	20	Members can create, read, update, and delete their own posts.
Image Upload	10	Admin can upload images to posts. Member profile pictures upload and display.
Comments / Replies	10	Members can add and delete their own comments on any post.
Profile Update	10	Name, bio, and profile picture can all be updated from the profile page.
Admin Dashboard	15	Admin can activate/deactivate members and remove inappropriate posts.
Role-Based Access	5	Guests cannot post. Members cannot access /admin. All ProtectedRoutes work.
TOTAL	100	

Congratulations on completing Phase 2! You have built a complete full-stack web application — with authentication, file uploads, a real database, and role-based access control. This is real-world, job-ready development experience.